國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab05

姓名：林彥廷

學號：112370219

Lab01-Q1.
Shuffling and Dealing Cards (Use Stack) **1. Complete Class Stack, shuffleDeck and drawALLCards in the sample code.** **2. Add comments to the program to clearly describe its actions.**
Code

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
using namespace std;


// 表示單張撲克牌的類別
class Card {
public:
    string colors; // 儲存撲克牌的花色
    string rank; // 儲存撲克牌的數值

    // 建立 constructor 來初始化物件,當 Card 物件建立時,它會自動執行這個函式,並把 s
和 r 的值存入 colors 和 rank
    Card(string s, string r) : colors(s), rank(r) {}

    void display() const { // 顯示撲克牌的資訊
        cout << rank << " of " << colors << endl;
    }
};

// 實作 Stack
class Stack {
private:
    vector<Card> stack; // 表示 stack 是一個能存放 Card 類別物件的 vector

public:
    void push(const Card& card) {
        stack.push_back(card); // 將打亂的牌放入 stack
    }
```

```
    Card pop() {
        if (!isEmpty()) {
            Card topcard = stack.back(); // 變數 topcard 儲存上面第一張牌。back()
是 std::vector 中取得最後一個元素的用法
            stack.pop_back(); // 將上面第一張牌從 stack 中刪除
            topcard.display(); // 利用 display 函式印出 topcard
            return topcard;
        }
        return Card("", ""); // 若 stack 為空，則回傳一張無數值的空牌
    }

    bool isEmpty() const {
        return stack.empty(); // 檢查 stack 是否為空並回傳
    }
};

// 代表一副撲克牌的類別
class Deck {
private:
    vector<Card> cards; // 存放未洗牌的撲克牌
    Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理

public:
    Deck() { // 建立 constructor 來初始化物件
        string colors[] = { "Hearts", "Diamonds", "Clubs", "Spades" }; // 儲存撲
克牌的花色

        string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9","10", "J",
"Q", "K", "A" }; // 儲存撲克牌的數值

        // 利用迴圈將 52 張牌加入 cards 這個 vector 裡面
        for (int i = 0; i < 4; i++) { // 四種花色
            for (int j = 0; j < 13; j++) { // 13 種點數
                cards.push_back(Card(colors[i], ranks[j]));
            }
        }
    }
```

```
    // 洗牌(Hint:使用 shuffle 函數)
    void shuffleDeck() {
        random_device rd; // 取得種子
        mt19937 g(rd()); // 產生隨機數產生器
        shuffle(cards.begin(), cards.end(), g); // 將 vector 打亂

        for (int i = 0; i < cards.size(); i++) {
            shuffledDeck.push(cards[i]); // 洗好的牌一張一張放入 stack
        }
    }

    // 發牌 pop
    void drawAllCards() {
        for (int i = cards.size(); i > 0; i--) {
            shuffledDeck.pop(); // 牌一張一張從 stack 發出
        }
    }
};


int main() {
    Deck deck; // 建立 deck 產生 52 張撲克牌
    deck.shuffleDeck(); // 進行洗牌並放入堆疊
    cout << "Shuffled deck:" << endl;
    deck.drawAllCards(); // 依序取出堆疊內的牌並顯示
    return 0;
}
```

Result

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
using namespace std;


// 表示單張撲克牌的類別
class Card {
public:
    string colors; // 儲存撲克牌的花色
    string rank; // 儲存撲克牌的數值

    // 建立 constructor來初始化物件,當 Card 物件建立時,它會自動執行這個函式,並把 s 和 r 的值存入colors 和 rank
    Card(string s, string r) : colors(s), rank(r) {}

    void display() const { // 顯示撲克牌的資訊
        cout << rank << " of " << colors << endl;
    }
};

// 實作 Stack
class Stack {
private:
    vector<Card> stack; // 表示 stack 是一個能存放 Card 類別物件的 vector

public:
    void push(const Card& card) {
        stack.push_back(card); // 將打亂的牌放入stack
    }

    Card pop() {
        if (!isEmpty()) {
            Card topcard = stack.back(); // 變數topcard 儲存上面第一張牌，back() 是std::vector 中取得最後一個元素的用法
            stack.pop_back(); // 將上面第一張牌從stack 中刪除
            topcard.display(); // 利用display 函式印出topcard
            return topcard;
        }
        return Card("", ""); // 若stack 為空，則回傳一張無數值的空牌
    }

    bool isEmpty() const {
        return stack.empty(); // 檢查stack 是否為空並回傳
    }
};

// 代表一副撲克牌的類別
class Deck {
private:
    vector<Card> cards; // 存放未洗牌的撲克牌
    Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理
```

```
Shuffled deck:
6 of Hearts
4 of Diamonds
A of Spades
8 of Clubs
3 of Diamonds
Q of Diamonds
7 of Diamonds
10 of Spades
3 of Spades
J of Hearts
4 of Hearts
K of Clubs
9 of Spades
10 of Hearts
2 of Diamonds
5 of Clubs
5 of Diamonds
A of Diamonds
4 of Spades
3 of Hearts
3 of Clubs
6 of Spades
7 of Clubs
2 of Hearts
7 of Spades
A of Hearts
9 of Diamonds
5 of Spades
Q of Hearts
10 of Diamonds
J of Spades
9 of Clubs
K of Spades
9 of Hearts
7 of Hearts
6 of Clubs
6 of Diamonds
A of Clubs
4 of Clubs
8 of Spades
J of Diamonds
8 of Diamonds
J of Clubs
8 of Hearts
10 of Clubs
2 of Spades
Q of Spades
K of Hearts
Q of Clubs
K of Diamonds
5 of Hearts
2 of Clubs
```

```
Shuffled deck:
10 of Diamonds
8 of Spades
8 of Hearts
7 of Diamonds
3 of Diamonds
A of Clubs
Q of Clubs
J of Spades
2 of Diamonds
A of Diamonds
9 of Spades
4 of Diamonds
J of Diamonds
3 of Spades
6 of Diamonds
7 of Clubs
2 of Clubs
Q of Spades
7 of Hearts
5 of Clubs
5 of Spades
A of Hearts
7 of Spades
9 of Diamonds
K of Hearts
10 of Spades
6 of Spades
Q of Hearts
8 of Diamonds
K of Spades
J of Hearts
3 of Hearts
A of Spades
6 of Clubs
J of Clubs
6 of Hearts
4 of Spades
8 of Clubs
9 of Clubs
9 of Hearts
4 of Hearts
5 of Hearts
2 of Spades
K of Diamonds
K of Clubs
5 of Diamonds
J of Hearts
10 of Clubs
3 of Clubs
4 of Clubs
10 of Hearts
Q of Diamonds
```

```
Shuffled deck:
K of Spades
2 of Spades
5 of Clubs
9 of Spades
6 of Hearts
4 of Spades
7 of Spades
5 of Spades
9 of Clubs
10 of Clubs
J of Clubs
8 of Hearts
Q of Diamonds
3 of Diamonds
8 of Diamonds
Q of Clubs
6 of Diamonds
Q of Hearts
7 of Hearts
6 of Clubs
7 of Clubs
K of Hearts
J of Hearts
10 of Diamonds
J of Diamonds
K of Clubs
3 of Spades
A of Hearts
4 of Diamonds
4 of Hearts
A of Spades
2 of Hearts
K of Diamonds
7 of Diamonds
4 of Clubs
9 of Hearts
J of Clubs
8 of Spades
9 of Diamonds
3 of Hearts
A of Clubs
10 of Hearts
10 of Spades
5 of Hearts
5 of Diamonds
2 of Clubs
6 of Spades
Q of Spades
J of Spades
2 of Diamonds
A of Diamonds
8 of Clubs
```